

CONVEX OPTIMIZATION PROJECT 2026

Convex Approaches for IoT Intrusion Detection

Scientific positioning around Linear SVM and RBF Kernel SVM on RT-IoT2022

Course Convex Optimization
Group Baptiste GRIMBAUM, Abdellah H'LALI, Nathan PEGE, Henri SU, Clement WAWSZCZYK
Dataset RT-IoT2022 - UCI Machine Learning Repository
Task Multi-class classification of IoT attacks

Contents

I	Scientific Positioning and Experimental Study	3
1	Abstract	3
2	Introduction and Scientific Motivation	3
3	Application Framework: Intrusion Detection in IoT Environments	3
4	RT-IoT2022 Dataset and Classification Problem Formulation	4
5	Mathematical Positioning: Why the SVM is Central to this Project	5
6	State of the Art Focused on RT-IoT2022 and SVM Models	5
7	Identified Gaps in the Literature	6
8	Problem Statement, Hypotheses, and Contributions	6
9	Experimental Study: Protocol, Results, and Analysis	6
9.1	Experimental Objective	6
9.2	Data Preparation	7
9.3	Class Distribution	7
9.4	Experimental Protocol	7
9.5	Cross-Validation Results	8
9.6	Final Evaluation on the Test Set	8
9.7	Timing Analysis	9
9.8	Per-Class Analysis	9
9.9	Confusion Matrices	10
9.10	Experimental Discussion	11
9.11	Experimental Limitations	11
II	Mathematical Analysis: Linear SVM and RBF SVM	11
10	Mathematical Analysis: Linear SVM	11
10.1	Introduction to the Linear SVM	11
10.1.1	Multi-class Strategy and Notation	11
10.2	Primal Problem Formulation	12
10.3	Analysis of Objective Function Components	12
10.3.1	The Regularization Term $\frac{1}{2}\ w\ _2^2$	12
10.3.2	The Extended Penalty Term (L2-loss)	13
10.4	Constraint Analysis	13
10.4.1	Absence of the Explicit Constraint $\xi_i \geq 0$	13
10.5	Validation of Slater's Condition	13
10.6	Construction of the Lagrangian	14
10.7	Karush-Kuhn-Tucker (KKT) Optimality Conditions	14
10.7.1	Stationarity	14
10.7.2	Feasibility and Complementarity	15
10.8	Establishment of the Dual Problem	15
10.9	Analytical Solution and Algorithmic Complexity	16
10.9.1	Intractability in High Dimensions	16
10.10	Return to Primal and the TRON Algorithm	17
10.10.1	Gradient and Generalized Hessian	17
10.10.2	Principle of TRON (<i>Trust Region Newton Method</i>)	17
10.10.3	Convergence Guarantees	17

11 Theoretical Formulation of the RBF SVM (Non-linear)	18
11.1 Quadratic Optimization Problem	18
11.2 Solving the Problem	19
11.2.1 Verifying Convexity	19
11.2.2 Verifying Slater's Condition	19
11.2.3 The Lagrangian	19
11.2.4 KKT Conditions	20
11.2.5 Transition to the Dual Problem and RBF Kernel Justification	20
11.2.6 Problem Encountered with the RBF	21
11.2.7 The SMO (Sequential Minimal Optimization) Method [14]	21
Conclusion	23
References	24

Part I

Scientific Positioning and Experimental Study

1 Abstract

This study is part of a convex optimization project applied to a concrete machine learning problem: the multi-class classification of network attacks in IoT environments. The selected dataset, RT-IoT2022, provides a relevant experimental framework because it combines normal traffic, several realistic attack families, and network variables derived from a real-time IoT infrastructure. Recent literature on RT-IoT2022 predominantly favors model comparisons based on classification scores, particularly using non-linear models, trees, ensembles, or deep architectures. In this context, our work focuses on a more specific question: understanding why the RBF kernel SVM is frequently used when a linear SVM more directly related to a convex formulation and theoretically more scalable could serve as a solid alternative. This document establishes the scientific context, describes the dataset, synthesizes the state of the art, and formalizes the problem prior to the experimental study.

Keywords: RT-IoT2022; IoT cybersecurity; intrusion detection system; linear SVM; RBF kernel; convex optimization; multi-class classification; class imbalance.

2 Introduction and Scientific Motivation

The widespread adoption of connected objects is transforming the attack surface of digital systems. In an IoT network, communications between sensors, gateways, cloud services, and embedded devices produce heterogeneous traffic that is often continuous and sometimes difficult to monitor with static rules. Denial-of-service attacks, network scans, ARP poisoning, or SSH brute force attempts can appear as variations in network flows: solicited ports, flow duration, packet volume, TCP flags, inter-arrival times, or asymmetry between upstream and downstream traffic.

Intrusion Detection Systems (IDS) address this problem by analyzing observed flows to distinguish normal behavior from malicious behavior. Supervised machine learning approaches are particularly suitable when annotated data is available: they transform a network security problem into a classification problem. In the case of RT-IoT2022, the target variable corresponds to the type of traffic or attack observed, allowing for the construction of a multi-class classifier.

However, the objective of this project is not solely to achieve a high score. Within the framework of a convex optimization course, the model must also be analyzed through its objective function, geometry, constraints, parameters, and optimization method. This leads us to favor models with an explicit mathematical structure. For this reason, the SVM is a central model: it directly links classification to the concepts of maximum margin, regularization, slack variables, and convex quadratic programming.

The starting point of our study is the tension between experimental practice and computational theory. Several works on RT-IoT2022 use or compare RBF kernel SVMs, often yielding good scores. Yet, kernelized implementations such as SVC are known to have a training cost that scales poorly with the number of samples. Conversely, the linear SVM is less flexible but more directly interpretable, closer to the convex formulation studied in class, and generally better suited for large datasets.

3 Application Framework: Intrusion Detection in IoT Environments

An IDS applied to IoT must satisfy two constraints that are not always compatible. The first is detection performance: missing an attack can have significant operational costs, especially in the case of a distributed attack or unauthorized access attempt. The second is the deployment constraint: in an IoT environment, available resources in terms of memory, computation, and energy are limited, especially if detection must be performed close to the data source.

This dual constraint explains why the analysis cannot be limited to accuracy alone. A model can achieve high overall performance by correctly recognizing majority classes while failing on rare but critical attacks. Furthermore, a highly performant model that is too costly to train or use for prediction may be impractical for a real-time IDS. Evaluation must therefore integrate per-class metrics, macro F1-score, recall, confusion matrices, training time, and prediction time.

In this study, we choose to retain a multi-class formulation. Binary classification (normal/attack) answers a simple operational question but masks the nature of the attack. Multi-class classification is more difficult, particularly in the presence of rare classes, but it provides more useful information for security analysis: it identifies whether a flow resembles an Nmap scan, a Slowloris attack, a SYN Hping attack, or an SSH brute force attempt.

4 RT-IoT2022 Dataset and Classification Problem Formulation

RT-IoT2022 is an IoT network traffic dataset introduced by Sharmila and Nagapadma, and subsequently distributed via the UCI Machine Learning Repository. The UCI page describes this dataset as originating from a real-time IoT infrastructure and covering both normal and adversarial behaviors. It also specifies that the data comes from devices such as ThingSpeak-LED, Wipro-Bulb, and MQTT-Temp, along with attack scenarios including Brute-Force SSH, DDoS, Slowloris, and Nmap patterns [1].

Network flows are described by numerical and categorical variables extracted using Zeek and Flowmeter. These variables cover several dimensions of the traffic: source and destination ports, protocol, service, flow duration, packet count, throughput, TCP flags, payload sizes, inter-arrival times, and active/idle periods. The value of this dataset lies in representing intrusion detection not as a simple abstract classification, but as a task directly tied to the observable properties of network traffic.

Table 1 - Main Characteristics of RT-IoT2022

Property	Value	Relevance to the study
Number of instances	123,117	Sufficient volume to discuss scalability.
Number of features	83	Feature space rich enough to compare linearity vs. kernel.
Feature types	Real and categorical	Requires coherent standardization and encoding.
Main task	Classification	Aligns with the IDS goal: identify traffic type.
Missing values	None (per UCI)	Simplifies preprocessing, though verification is still needed.
Target variable	Attack_type	Multi-class formulation: normal traffic or attack family.

We denote the dataset $D = \{(x_i, y_i)\}_{i=1}^n$, where x_i represents the feature vector associated with network flow i and y_i belongs to the set of attack classes or normal traffic. The problem studied is thus to learn a decision function f capable of mapping a new flow x to a predicted class among K possible classes. This formulation is deliberately general: it allows for the subsequent comparison of a linear model, where the boundary is built in the original feature space, and an RBF model, where the separation is produced implicitly in a higher-dimensional feature space induced by a kernel.

5 Mathematical Positioning: Why the SVM is Central to this Project

The choice of the SVM is motivated not only by its recurrent presence in applied literature but also by its intrinsic relevance to the study of convex optimization. The **soft-margin linear SVM** serves as an ideal textbook case: its learning is based on minimizing a strictly convex quadratic objective function subject to affine inequality constraints. This model rigorously analyzes the trade-off between maximizing the geometric margin (via the regularization term) and tolerating classification errors (via slack variables controlled by a penalty parameter).

Conversely, the **RBF kernel SVM** bypasses the linearity assumption by evaluating the local similarity metric between network flows. Using a Gaussian kernel, it implicitly projects observations into a higher-dimensional feature space, allowing for curved decision boundaries. While this flexibility often yields excellent empirical scores, it drastically changes the optimization structure: the resolution shifts to a dual formulation dependent on the Gram matrix, whose spatial and temporal complexity scales at least quadratically with the number of samples.

Faced with over 123,000 instances in the RT-IoT2022 dataset, the confrontation between these two variants illustrates the fundamental trade-off in statistical learning between **geometric expressivity** and **computational scalability**. Our study leverages this tension to illustrate course concepts: the detailed mathematical analysis (formulations, shift to the dual, KKT conditions, and solver behavior) is deferred to Part 2 of the report.

6 State of the Art Focused on RT-IoT2022 and SVM Models

Recent state-of-the-art research around RT-IoT2022 shows growing interest in IoT intrusion detection. However, a clear trend emerges: existing works primarily evaluate experimental pipelines and classification scores, often treating models as black boxes without linking observed performance to mathematical optimization properties. Table 2 summarizes the major contributions from the literature utilized in our study and highlights both their contributions and their limitations regarding our analytical angle.

Table 2 - Summary of Referenced Works and Positioning of Our Study

Reference	Approach Studied	Contribution to Our Project	Identified Limitation
Sharmila & Nagapadma, 2023 [2]	QAE for IDS on RT-IoT2022	Introduces the dataset and the constrained IoT context.	Non-convex model; no SVM analysis.
Sharmila et al., 2024 [3]	Statistical tests + Linear/RBF SVM, KNN, NB, DT	Shows a direct linear/RBF comparison on RT-IoT2022.	Mostly statistical analysis, not focused on convex optimization.
Kumar et al., 2025 [5]	Mutual Information + classifiers including SVM	Illustrates the applied use of RBF SVM after preprocessing.	SVM treated as an experimental component; formulation not developed.
Dymora et al., 2026 [4]	Multi-class comparison of ML models on RT-IoT2022	Reinforces the need for a protocol with metrics and timing.	Kernel cost observed but loosely linked to dual and support vectors.
scikit-learn Documentation [6,7]	SVC, LinearSVC, SVM complexity	Provides technical arguments on scalability.	Technical documentation, not an RT-IoT2022 specific study.

This summary shows that our project does not seek to add yet another model to a score-chasing competition. It uses the comparison between linear and RBF SVMs as an entry point into a deep discussion of optimization: shape of the objective function, convexity, the role of regularization, multi-class strategy, kernel cost (Gram matrix), and the geometric interpretation of support vectors.

7 Identified Gaps in the Literature

Our review of the state of the art highlights four major gaps that this project aims to fill:

- **Optimization relegated to the background:** Applied works describe pipelines and metrics but omit the underlying mathematical problem. Concepts like primal, dual, slack variables, and KKT conditions are often absent.
- **Computational cost ignored:** The trade-off between the RBF kernel’s geometric flexibility and its training cost is rarely discussed, despite the dataset’s large number of observations ($n > 123,000$).
- **Evaluation biased by imbalance:** The predominant use of global accuracy masks detection errors on rare attacks. Reliable operational analysis requires macro F1-scores and confusion matrices.
- **Underutilization of the linear SVM:** Although it is the most direct reference model linking experimentation to theory (minimizing a convex function, margin interpretation), it is rarely used as a central object of mathematical analysis.

8 Problem Statement, Hypotheses, and Contributions

The core problem of the study can be formulated as follows:

Why do IoT intrusion detection studies frequently favor the RBF kernel SVM when a linear SVM seems theoretically more suited for large datasets? On RT-IoT2022, can the linear SVM offer a more relevant compromise between classification performance, computational cost, interpretability, and convex analysis?

This question will be examined through four working hypotheses:

H1 - RBF Flexibility: The RBF SVM may achieve better performance on certain classes because it constructs a non-linear boundary, better suited to complex distributions.

H2 - Linear SVM Compromise: The linear SVM may remain competitive if the network variables already encode a sufficiently informative separation between classes.

H3 - Importance of Cost: A moderate performance gap is not enough to justify an RBF model if its training or prediction cost is significantly higher.

H4 - Mathematical Relevance: The linear SVM is the best foundation for a comprehensive convex optimization analysis, while the RBF model introduces the shift to the dual and the role of the kernel.

The expected contributions are twofold. Experimentally, the study provides a fair comparison between Linear SVM and RBF SVM on the same dataset, with the same preprocessing, train/test split, stratified cross-validation, and metrics. Mathematically, it paves the way for an analysis of the linear SVM’s convex formulation and the shift to the RBF kernel, preventing the project from being reduced to a simple scikit-learn implementation exercise.

9 Experimental Study: Protocol, Results, and Analysis

9.1 Experimental Objective

This part compares two variants of SVM for multi-class classification of IoT attacks in RT-IoT2022: a linear SVM trained using LinearSVC, and an RBF kernel SVM trained using SVC(kernel="rbf"). The objective is not solely to maximize accuracy, but to evaluate detection quality on a highly imbalanced dataset and to observe the operational cost of both model families.

In an IDS, rare classes cannot be ignored in favor of the majority class. Macro metrics, particularly macro F1 and macro recall, are therefore essential: they give equal weight to each class. Per-sample prediction time completes the analysis, as a detection system must remain usable on continuous network flows.

9.2 Data Preparation

The dataset is loaded from the RT_IOT2022 file. The notebook detects 123,117 observations and 85 initial columns. The target column is Attack_type. The technical column Unnamed: 0 is dropped, leaving 83 explanatory variables after X/y separation. No missing values and no duplicates are detected. The constant column bwd_URG_flag_count is flagged by the notebook, but not automatically dropped.

The train/test split is stratified using test_size=0.2 and random_state=42. Both models then use the same preprocessing: StandardScaler for the 81 numerical variables and OneHotEncoder(handle_unknown='ignore') for the two categorical variables, proto and service.

Element	Result	Interpretation
Initial observations	123,117	Sufficient volume to study scalability
Initial columns	85	Including target and one technical column dropped
Features	83	81 numerical and 2 categorical
Missing values	0	No imputation necessary
Duplicates	0	No row deletion
Train / Test	98,493 / 24,624	Stratified split, independent final test

9.3 Class Distribution

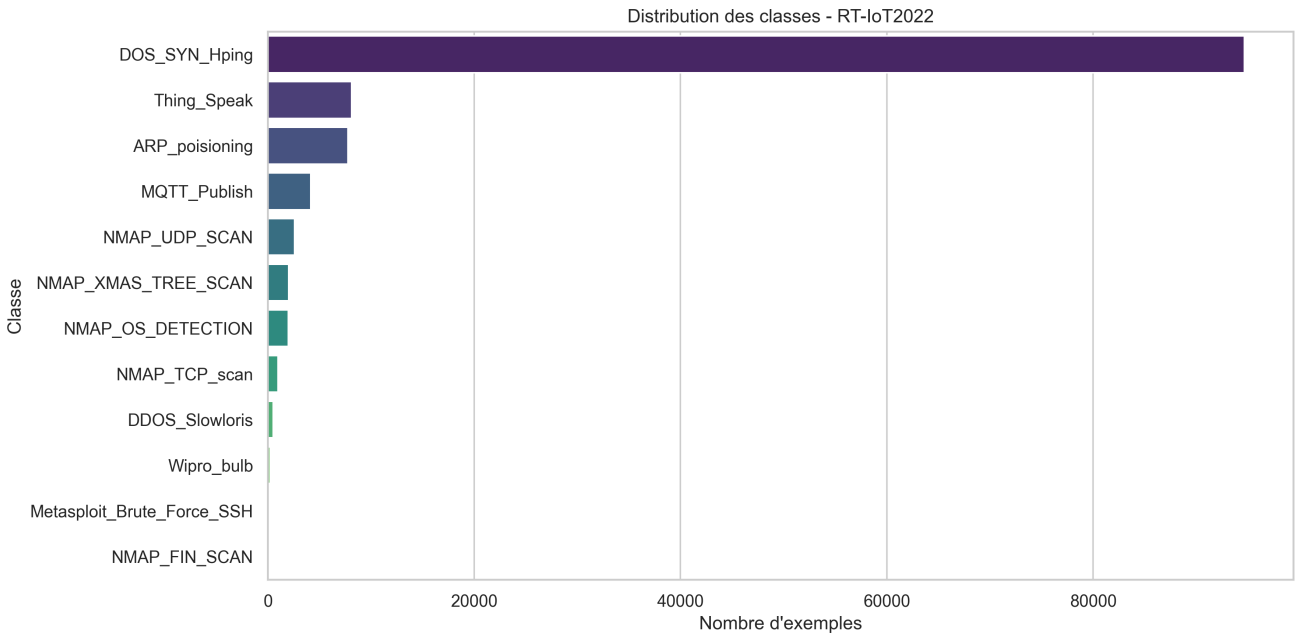


Figure 1: Class distribution of the RT-IoT2022 dataset.

The distribution is highly imbalanced: DOS_SYN_Hping accounts for 94,659 observations, or 76.89% of the dataset. The rarest classes are Metasploit_Brute_Force_SSH (37 observations) and NMAP_FIN_SCAN (28 observations). A high accuracy can therefore mask errors on underrepresented attacks; macro F1 and macro recall are necessary to evaluate inter-class balance.

9.4 Experimental Protocol

Hyperparameters are selected via stratified cross-validation using StratifiedKFold(n_splits=3, shuffle=True, random_state=42). The selection criterion is macro F1. The Linear SVM tests $C \in \{0.1; 1\}$. The RBF SVM tests $C \in \{1; 10\}$ and $\gamma \in \{\text{scale}; 0.01\}$. Final metrics are calculated on the same test set of 24,624 observations.

In the final version of the notebook, the RBF SVM is validated and trained on the full training set of 98,493 observations. Training times are therefore comparable in terms of data volume with those of the Linear SVM, even if the underlying implementations differ.

Training and prediction times are measured using `time.perf_counter()`. The primary variants do not use `class_weight="balanced"`; the notebook only contains a supplementary analysis with weighting, which is not included in the main tables below.

9.5 Cross-Validation Results

Model	Best Parameters	CV Samples	CV Accuracy	CV Macro F1	CV Macro Recall	CV Time
Linear SVM	C=1.0	98,493	0.9904	0.9335	0.9131	870.47 s
RBF SVM	C=10, gamma=0.01	98,493	0.9946	0.9463	0.9191	6.69 s

In cross-validation, the RBF SVM achieves the best macro F1 (0.9463) and the best average accuracy (0.9946). The gap remains small, however: the Linear SVM already reaches 0.9335 in macro F1, confirming that a linear boundary remains highly competitive with this preprocessing.

9.6 Final Evaluation on the Test Set

Model	Parameters	Train	Acc.	Prec.	Recall	F1	F1 w.	Train (s)	Pred. (ms)
Linear SVM	C=1; cw=None	98493	0.9913	0.9767	0.9177	0.9431	0.9912	979.62	0.000877
RBF SVM	C=10; gamma=.01; cw=None	98493	0.9952	0.9772	0.9277	0.9490	0.9951	2.13	0.149077

Table 5: Performance comparison between Linear SVM and RBF kernel SVM.

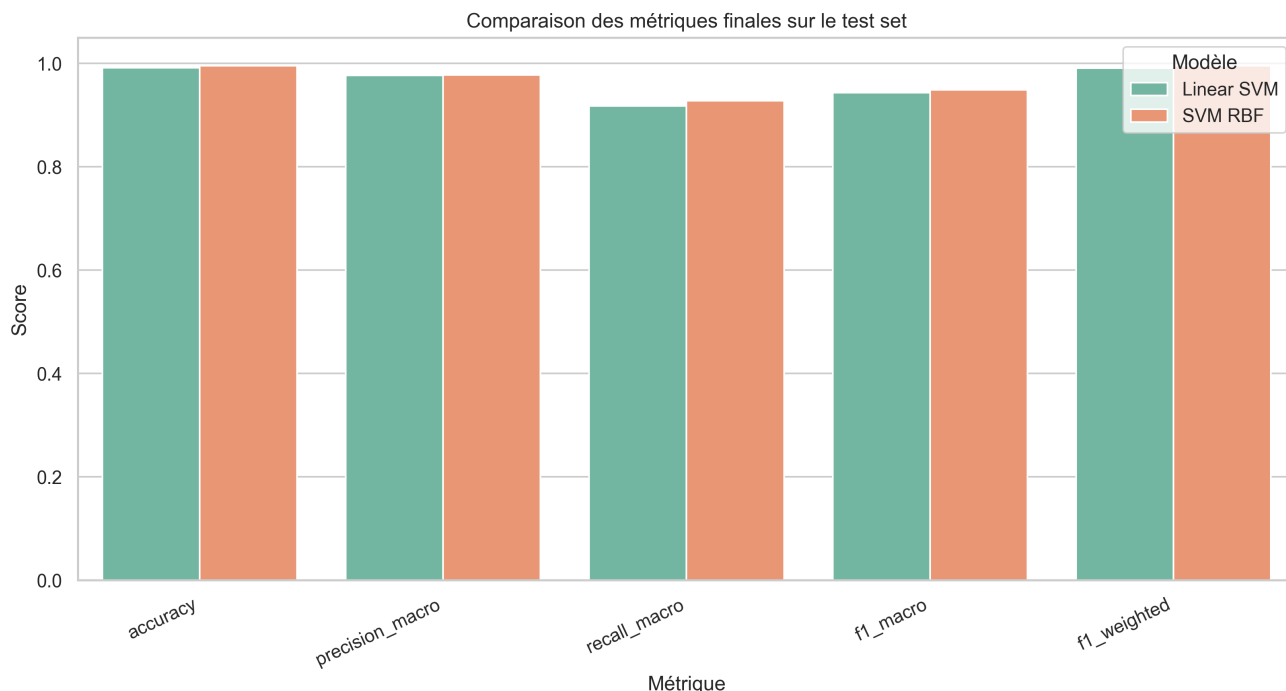


Figure 2: Comparison of final metrics on the test set.

On the test set, the RBF SVM is ahead in accuracy (0.9952 vs 0.9913), macro recall (0.9277 vs 0.9177), and macro F1 (0.9490 vs 0.9431). The weighted F1 is very high for both models because it strongly reflects the majority class; macro F1 is more discriminant for judging minority classes.

9.7 Timing Analysis

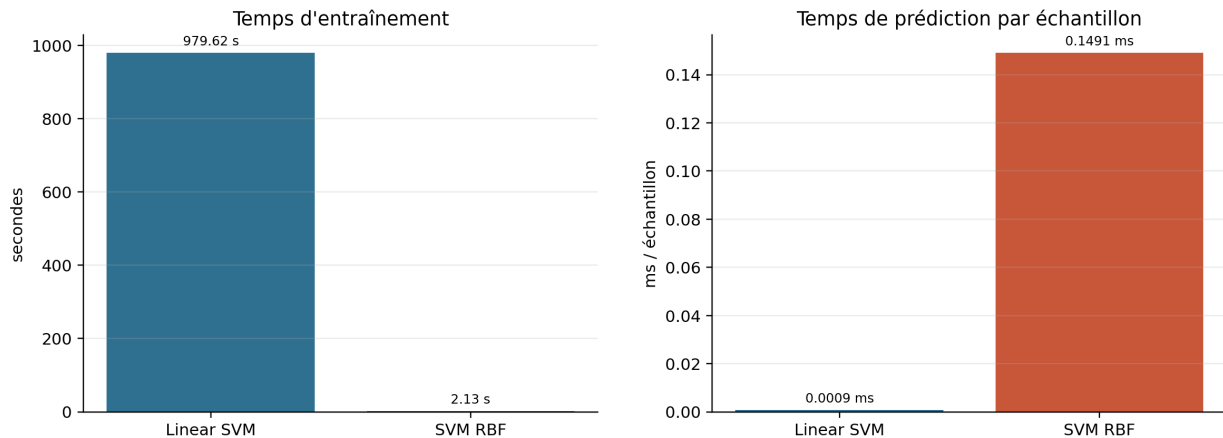


Figure 3: Comparison of training and prediction times.

Both models are trained on the same final volume of data: 98,493 observations. The observed training time is much higher for LinearSVC (979.62 s) than for the RBF SVM (2.13 s) in this execution. **This result should be interpreted as an empirical behavior of this specific configuration, not as a general reversal of theoretical complexities.**

Prediction time is directly comparable as it is measured on the same test set. The RBF SVM requires 0.149077 ms per sample compared to 0.000877 ms for the linear model; it is therefore approximately 170.1 times slower per sample on the test set.

9.8 Per-Class Analysis

Class	Test Support	Linear Recall	Linear F1	RBF Recall	RBF F1	F1
DOS_SYN_Hping	18,932	1.0000	1.0000	1.0000	1.0000	+0.0000
Thing_Speak	1,622	0.9642	0.9496	0.9741	0.9780	+0.0284
ARP_poisoning	1,550	0.9355	0.9477	0.9858	0.9726	+0.0249
MQTT_Publish	829	0.9940	0.9970	0.9928	0.9958	-0.0012
NMAP_UDP_SCAN	518	0.9826	0.9586	0.9846	0.9724	+0.0138
NMAP_XMAS_TREE_SCAN	402	0.9950	0.9963	0.9950	0.9975	+0.0012
NMAP_OS_DETECTION	400	1.0000	0.9938	1.0000	0.9988	+0.0050
NMAP_TCP_scan	200	1.0000	0.9975	1.0000	0.9975	+0.0000
DDOS_Slowloris	107	0.8037	0.8776	0.8037	0.8912	+0.0136
Wipro_bulb	51	0.6471	0.7674	0.7059	0.8276	+0.0601
Metasploit_Brute_Force_SSH	7	0.8571	0.9231	0.8571	0.9231	+0.0000
NMAP_FIN_SCAN	6	0.8333	0.9091	0.8333	0.8333	-0.0758

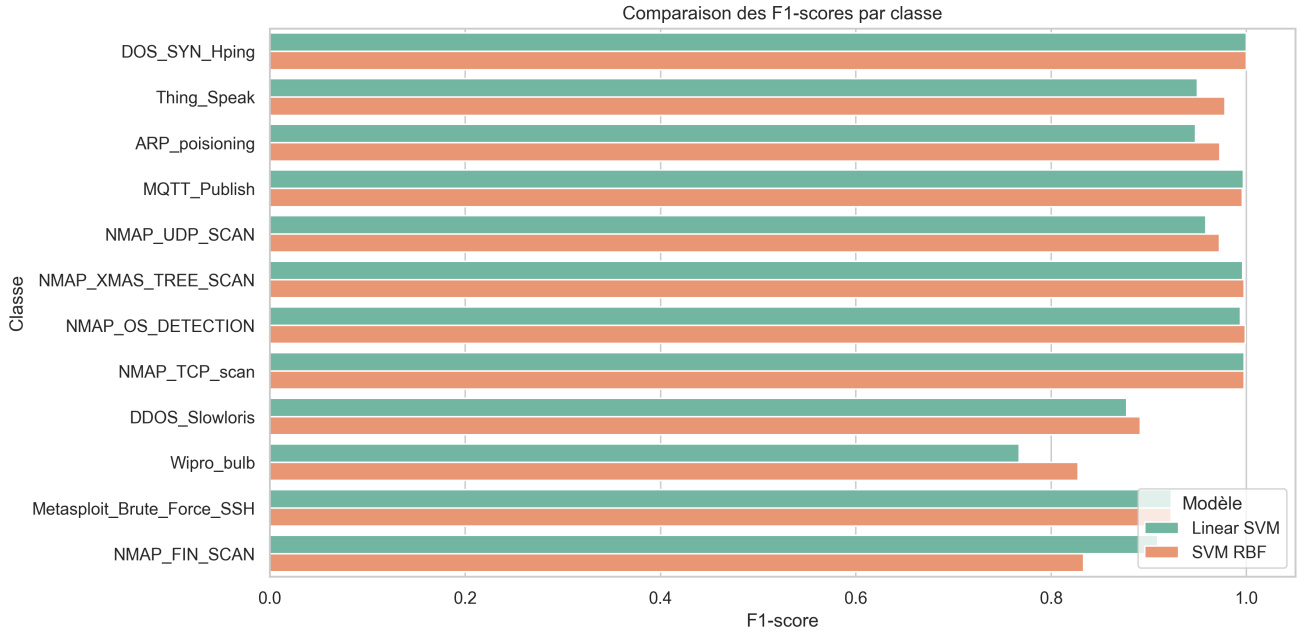


Figure 4: Comparison of F1 scores per class.

Both models perfectly recognize DOS_SYN_Hping on the test set. The RBF model mostly improves Wipro_bulb (+0.0601 in F1), Thing_Speak (+0.0284), ARP_poisoning (+0.0249), and NMAP_UDP_SCAN (+0.0138). The Linear SVM is slightly better on MQTT_Publish and noticeably better on NMAP_FIN_SCAN in F1, but this last class only contains 6 examples in the test set; interpretation must therefore remain cautious.

9.9 Confusion Matrices

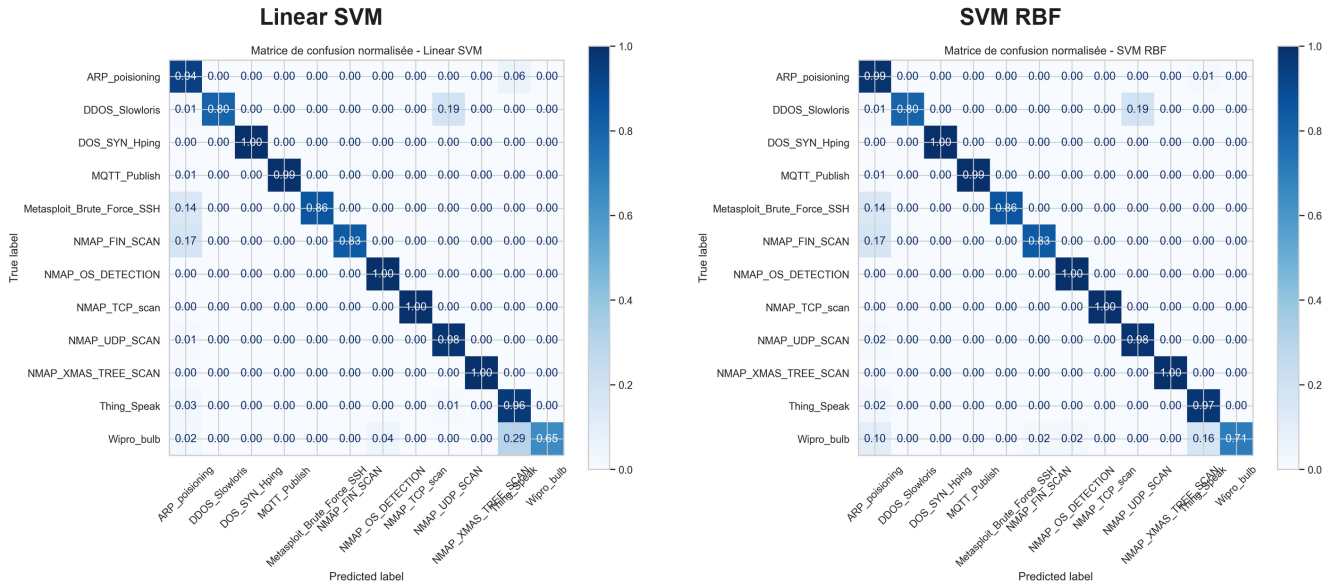


Figure 5: Normalized confusion matrices of both models.

The normalized matrices confirm that the dominant classes and several NMAP classes are very well recognized. The main limitations concern low-support classes, notably Wipro_bulb, DDOS_Slowloris, NMAP_FIN_SCAN, and Metasploit_Brute_Force_SSH. Errors on these classes weigh little on overall accuracy, but they strongly influence macro metrics.

9.10 Experimental Discussion

H1 is partially confirmed: the RBF kernel provides a measurable gain on the final macro F1 and improves several classes, notably ARP_poisoning, Thing_Speak, and Wipro_bulb. H2 is also confirmed: the Linear SVM remains highly competitive, with a 0.9431 macro F1 and a 0.9913 final accuracy.

H3 calls for a nuanced answer. In this execution, the RBF model is slower in prediction per sample, but its measured training is shorter than that of the Linear SVM. The theoretical complexity of the RBF is less favorable, but the observed behavior on RT-IoT2022 depends on the dataset's structure, hyperparameters, implementation, and the effective number of support vectors.

H4 is justified by these results: the performances are close, but the trade-offs differ. It is therefore relevant to mathematically study the role of the margin, C, slack variables, the dual, the RBF kernel, the Gram matrix, and support vectors.

9.11 Experimental Limitations

The experiment relies on a single stratified train/test split. Extremely rare classes, with 6 or 7 examples in the test set, make certain per-class gaps unstable. The hyperparameter grid is deliberately limited by FAST_MODE=True, using 3 folds and few C and gamma values. The main models do not use class_weight="balanced", even though the notebook explores this variant supplementarily. Finally, the measured times depend on the machine, software versions, and scikit-learn implementation.

Conclusion and Transition

In conclusion, this initial phase of empirical analysis highlights the relative strengths of each approach. While the RBF SVM excels at identifying minority classes at the cost of longer prediction times, the linear SVM stands out as a fast and fairly robust estimator for real-time applications. To grasp the fundamentals of this computational and geometric dynamics, we must now turn to a rigorous mathematical analysis of their respective dual formulations.

Part II

Mathematical Analysis: Linear SVM and RBF SVM

10 Mathematical Analysis: Linear SVM

10.1 Introduction to the Linear SVM

Faced with the binary subproblems induced by the One-vs-Rest strategy in the multi-class RT-IoT2022 task, the linear SVM seeks to determine an optimal separating hyperplane in a transformed feature space of dimension d , where $d = 92$ after standardization and one-hot encoding of the original features. This boundary meets two fundamental requirements:

1. **Margin Maximization:** The algorithm maximizes the distance between the boundary and the closest points of the two classes (support vectors), to ensure model robustness.
2. **Error Tolerance (Soft Margin):** To handle the class overlap inherent in real-world data, the soft margin allows violations of the separation constraints in exchange for a penalty in the objective function.

10.1.1 Multi-class Strategy and Notation

Generalization to the multi-class case relies on the *One-vs-Rest* (OvR) strategy, which breaks the problem down into K independent binary subproblems. For each classifier $k \in \{1, \dots, K\}$, the algorithm learns an hyperplane defined by a pair $(w^{(k)}, b^{(k)})$. For simplicity, the index (k) is omitted hereafter. The binary label associated with sample i is denoted $\tilde{y}_i \in \{-1, 1\}$.

10.2 Primal Problem Formulation

Under the selected options (`loss='squared_hinge'`, `penalty='l2'`), the model solves a constrained optimization problem soft-margin SVM with squared hinge loss, or *L2-regularized L2-loss SVC*:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^n C_i \xi_i^2$$

subject to the constraints:

$$\tilde{y}_i (w^\top x_i + b) \geq 1 - \xi_i \quad \forall i \in \{1, \dots, n\}$$

The parameters correspond to the following dimensions:

- $w \in \mathbb{R}^{92}$: weight vector orthogonal to the hyperplane.
- $b \in \mathbb{R}$: bias defining the distance of the hyperplane to the origin.
- $\xi = [\xi_1, \dots, \xi_n]^\top \in \mathbb{R}^n$: slack variables measuring classification error.
- $n = 123,169$: total number of observations in the RT-IoT2022 dataset.
- $C_i = C$: common penalty parameter applied to all observations in the main experiment, since `class_weight=None`. In this setting, all training samples contribute with the same regularization-error trade-off, regardless of their class frequency. i .

10.3 Analysis of Objective Function Components

10.3.1 The Regularization Term $\frac{1}{2} \|w\|_2^2$

The vector w parameterizes the orientation of the separation boundary. Its L2 norm is defined by:

$$\|w\|_2 = \sqrt{\sum_{j=1}^{92} w_j^2}$$

Relationship between Norm Minimization and Margin Maximization. Consider the two parallel marginal hyperplanes:

$$H_1 : w^\top x + b = 1 \quad (\text{for } \tilde{y}_i = 1)$$

$$H_2 : w^\top x + b = -1 \quad (\text{for } \tilde{y}_i = -1)$$

The unit vector normal to these planes is $\frac{w}{\|w\|_2}$. The total geometric margin γ corresponds to the projection of the vector $(x_1 - x_2)$ (where $x_1 \in H_1$ and $x_2 \in H_2$) onto this normal vector:

$$\begin{aligned} \gamma &= (x_1 - x_2)^\top \frac{w}{\|w\|_2} \\ &= \frac{w^\top x_1 - w^\top x_2}{\|w\|_2} \\ &= \frac{(1 - b) - (-1 - b)}{\|w\|_2} \\ &= \frac{2}{\|w\|_2} \end{aligned}$$

Maximizing the margin γ is therefore equivalent to minimizing $\|w\|_2$. For convenience during derivation, the objective is formulated in the convex form $\frac{1}{2} \|w\|_2^2$.

Convexity Analysis of the Regularization Term. Let $f(w) = \frac{1}{2}\|w\|_2^2$. Its gradient and Hessian matrix are:

$$\nabla f(w) = w, \quad \nabla^2 f(w) = I_{92}$$

Since the Hessian matrix is the identity matrix, all its eigenvalues are $1 > 0$: it is positive definite, demonstrating the **strict convexity** of this term and guaranteeing the uniqueness of the global minimum.

10.3.2 The Extended Penalty Term (L2-loss)

This second term quantifies and penalizes margin violations via the squared hinge loss. At the optimum, each slack variable satisfies:

$$\xi_i = \max(0, 1 - \tilde{y}_i(w^\top x_i + b))$$

Squaring (ξ_i^2) induces a specific behavior compared to the standard SVM (L1-loss): minor violations ($\xi_i < 1$) have a small impact, while major errors ($\xi_i \gg 1$) incur a severe quadratic penalty. In cybersecurity, this choice forces the model to prioritize correcting attacks that deviate significantly from legitimate traffic profiles.

Let $P(\xi) = \sum_{i=1}^n C_i \xi_i^2$. Its Hessian matrix is diagonal:

$$\nabla^2 P(\xi) = \text{diag}(2C_1, 2C_2, \dots, 2C_n)$$

Since the eigenvalues $\lambda_i = 2C_i$ are all strictly positive ($C > 0$), the penalty function is also **strictly convex**.

10.4 Constraint Analysis

Geometrically, the state of an observation i breaks down into three cases:

1. **Inactive constraint** ($\tilde{y}_i(w^\top x_i + b) > 1$): the observation is correctly classified beyond the margin, $\xi_i = 0$.
2. **Active constraint on the margin** ($\tilde{y}_i(w^\top x_i + b) = 1$): the observation is on the marginal boundary, $\xi_i = 0$; this point is a *support vector*.
3. **Active constraint with violation** ($\tilde{y}_i(w^\top x_i + b) < 1$): the margin is violated, $\xi_i = 1 - \tilde{y}_i(w^\top x_i + b) > 0$.

10.4.1 Absence of the Explicit Constraint $\xi_i \geq 0$

Unlike the classic SVM (L1-loss), the L2-loss formulation does not explicitly require the $\xi_i \geq 0$ constraint. Because the penalty term is quadratic (ξ_i^2), any negative value of ξ_i would unnecessarily increase the objective function without helping to satisfy the inequality constraint: positivity at the optimum is implicitly guaranteed.

Since the constraint function $g_i(w, b, \xi_i) = 1 - \tilde{y}_i(w^\top x_i + b) - \xi_i$ is affine with respect to primal variables, it is both convex and concave, validating the regularity of constraints for convex optimization.

10.5 Validation of Slater's Condition

To ensure that the mathematical tools of duality apply correctly, we must verify Slater's condition. It requires the existence of at least one realistic starting point that lies strictly within the interior of the feasible domain (meaning it strictly satisfies all inequalities with $g_i < 0$). Since we are allowed to choose any starting point to prove this, let's simplify our lives and choose a deliberately trivial case where the hyperplane and the bias are zero:

$$w_0 = 0 \in \mathbb{R}^d$$

$$b_0 = 0 \in \mathbb{R}$$

If we inject this starting point into our constraint function g_i , the entire part related to the hyperplane vanishes:

$$1 - \tilde{y}_i(0^\top x_i + 0) = 1 - 0 = 1$$

The constraint to validate for each example i then reduces to:

$$1 - \xi_{i,0} < 0 \implies \xi_{i,0} > 1$$

Now, we simply need to construct a vector ξ_0 by choosing a value strictly greater than 1 for all rows. For instance, let's choose to set all errors to 2:

$$\xi_0 = [2, 2, \dots, 2]^\top \in \mathbb{R}^n$$

Let's substitute everything back into our original constraint function to verify the result:

$$g_i(0, 0, 2) = 1 - 0 - 2 = -1$$

Since $-1 < 0$, the strict inequality $g_i(w_0, b_0, \xi_0) < 0$ is perfectly satisfied for all rows in the dataset. This simple construction demonstrates that our search space is not empty and that a strictly feasible point does indeed exist. Slater's condition is therefore validated, which guarantees that strong duality relations will apply without any issues.

10.6 Construction of the Lagrangian

We introduce the Lagrange multiplier vector $\alpha = [\alpha_1, \dots, \alpha_n]^\top \in \mathbb{R}^n$, with $\alpha_i \geq 0$. The Lagrangian is written as:

$$L(w, b, \xi, \alpha) = \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^n C_i \xi_i^2 - \sum_{i=1}^n \alpha_i [\tilde{y}_i(w^\top x_i + b) - 1 + \xi_i]$$

By expanding and grouping the terms by variable, the formula becomes more usable:

$$L(w, b, \xi, \alpha) = \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^n C_i \xi_i^2 - \sum_{i=1}^n \alpha_i \tilde{y}_i w^\top x_i - b \sum_{i=1}^n \alpha_i \tilde{y}_i + \sum_{i=1}^n \alpha_i - \sum_{i=1}^n \alpha_i \xi_i$$

10.7 Karush-Kuhn-Tucker (KKT) Optimality Conditions

With Slater's condition verified, the optimal point $(w^*, b^*, \xi^*, \alpha^*)$ necessarily satisfies the KKT conditions.

10.7.1 Stationarity

Setting the partial derivatives of the Lagrangian to zero gives:

- **With respect to the weight vector w :**

$$\nabla_w L = w - \sum_{i=1}^n \alpha_i \tilde{y}_i x_i = 0 \implies w^* = \sum_{i=1}^n \alpha_i \tilde{y}_i x_i$$

- **With respect to the bias b :**

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i \tilde{y}_i = 0 \implies \sum_{i=1}^n \alpha_i \tilde{y}_i = 0$$

- **With respect to the slack variables ξ_i :**

$$\frac{\partial L}{\partial \xi_i} = 2C_i \xi_i - \alpha_i = 0 \implies \xi_i^* = \frac{\alpha_i}{2C_i} \quad \text{or} \quad \alpha_i = 2C_i \xi_i^*$$

The condition $\nabla_w \mathcal{L} = 0$ shows that the optimal weight vector is a linear combination of the support vectors ($\alpha_i > 0$). The expression for ξ_i^* is specific to the L2-loss formulation: unlike standard SVM where α_i is upper-bounded by C , the relationship between the dual variable and the error is purely linear here.

10.7.2 Feasibility and Complementarity

The KKT conditions also impose:

- **Dual feasibility:** $\alpha_i \geq 0 \forall i$.
- **Primal feasibility:** $\tilde{y}_i(w^{*\top} x_i + b^*) \geq 1 - \xi_i^* \forall i$.
- **Complementary slackness:** $\alpha_i [\tilde{y}_i(w^{*\top} x_i + b^*) - 1 + \xi_i^*] = 0 \forall i$.

If an observation is correctly classified outside the margin, its constraint is strictly satisfied, implying $\alpha_i = 0$.

10.8 Establishment of the Dual Problem

Here is the step-by-step breakdown of how each component transforms and simplifies:

1. **Simplification of the weight terms (w):** The parts involving the weight vector, $\frac{1}{2}\|w\|_2^2 - \sum_{i=1}^n \alpha_i \tilde{y}_i w^\top x_i$, combine and collapse. By substituting $w^* = \sum_{j=1}^n \alpha_j \tilde{y}_j x_j$, the first term becomes:

$$\frac{1}{2}\|w\|_2^2 = \frac{1}{2}(w^*)^\top w^* = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i \tilde{y}_i x_i \right)^\top \left(\sum_{j=1}^n \alpha_j \tilde{y}_j x_j \right) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j$$

Similarly, substituting w^* into the linear term yields:

$$- \sum_{i=1}^n \alpha_i \tilde{y}_i w^\top x_i = - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j$$

Combining these two results produces a single, consolidated quadratic expression:

$$\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j$$

2. **Elimination of the bias term (b):** The term containing the bias, $-b \sum_{i=1}^n \alpha_i \tilde{y}_i$, vanishes completely from the equation. This is guaranteed by the equilibrium constraint $\sum_{i=1}^n \alpha_i \tilde{y}_i = 0$, which forces the entire coefficient of b to equal zero.
3. **Simplification of the slack variable terms (ξ):** The block containing the regularized error and slack variables simplifies by substituting the optimal slack value $\xi_i^* = \frac{\alpha_i}{2C_i}$:

$$\sum_{i=1}^n C_i \left(\frac{\alpha_i}{2C_i} \right)^2 - \sum_{i=1}^n \alpha_i \left(\frac{\alpha_i}{2C_i} \right) = \sum_{i=1}^n \frac{\alpha_i^2}{4C_i} - \sum_{i=1}^n \frac{\alpha_i^2}{2C_i} = - \sum_{i=1}^n \frac{\alpha_i^2}{4C_i}$$

This is formulated as a convex quadratic maximization problem constrained only by the dual variables:

$$\max_{\alpha \geq 0} D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j x_i^\top x_j - \sum_{i=1}^n \frac{\alpha_i^2}{4C_i}$$

subject to the linear equality constraint inherited from the bias derivative:

$$\sum_{i=1}^n \alpha_i \tilde{y}_i = 0$$

To align our formulation with standard reference equations (such as Appendix A.2 of the official LIBLINEAR solver documentation), we can condense the dual objective function into a compact matrix form. To do this, we define a modified kernel matrix $\bar{Q} \in \mathbb{R}^{n \times n}$, whose elements are defined as:

$$\bar{Q}_{ij} = \tilde{y}_i \tilde{y}_j x_i^\top x_j + \frac{1}{2C_i} \delta_{ij}$$

where δ_{ij} represents the Kronecker delta function, defined as:

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

This algebraic modification means that the off-diagonal elements ($i \neq j$) remain purely the label-weighted dot products $\tilde{y}_i \tilde{y}_j x_i^\top x_j$. On the main diagonal ($i = j$), however, a regularizing term $\frac{1}{2C_i}$ is added to the dot product. This adjustment perfectly absorbs the final penalty term of the objective function ($-\sum_{i=1}^n \frac{\alpha_i^2}{4C_i}$) into the quadratic form.

By introducing an all-ones vector $e = [1, 1, \dots, 1]^\top \in \mathbb{R}^n$, the linear summation of the dual variables can be written as a vector dot product:

$$\sum_{i=1}^n \alpha_i = 1\alpha_1 + 1\alpha_2 + \dots + 1\alpha_n = e^\top \alpha$$

Similarly, the combination of the double summation terms compresses into a single quadratic form:

$$\frac{1}{2} \alpha^\top \bar{Q} \alpha = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \bar{Q}_{ij}$$

Consequently, the Wolfe dual problem can be expressed in an elegant, standard matrix notation:

$$\max_{\alpha \geq 0} \quad e^\top \alpha - \frac{1}{2} \alpha^\top \bar{Q} \alpha$$

10.9 Analytical Solution and Algorithmic Complexity

If we temporarily ignore the non-negativity constraint ($\alpha \geq 0$) and the linear equality constraint to find the unconstrained maximum, we can compute the gradient of this objective function with respect to the vector α and set it to zero:

$$\nabla_\alpha \left(e^\top \alpha - \frac{1}{2} \alpha^\top \bar{Q} \alpha \right) = 0 \implies e - \bar{Q} \alpha = 0$$

Solving this linear system yields the explicit analytical optimal solution for the dual variables:

$$\alpha^* = \bar{Q}^{-1} e$$

10.9.1 Intractability in High Dimensions

Although exact, this direct resolution is inapplicable on RT-IoT2022 ($n \approx 120,000$, $d = 92$) for two reasons:

- **Spatial Complexity:** storing the dense matrix $\bar{Q}$ in double precision requires $120,000 \times 120,000 \times 8 \approx 115.2$ GB of RAM.
- **Time Complexity:** inverting $\bar{Q}$ via direct methods (Cholesky, Gauss) presents a complexity of $O(n^3) \approx 1.73 \times 10^{15}$ operations, which is computationally infeasible within a reasonable timeframe.

10.10 Return to Primal and the TRON Algorithm

When $n \gg d$, transitioning to the dual transforms an initially simple problem (92 variables) into an excessively heavy n -dimensional system. It is therefore preferable to solve directly in the primal space. Substituting the expression for ξ_i^* back into the primal problem, the inequality constraints disappear, resulting in the unconstrained formulation used by the LIBLINEAR solver:

$$\min_{w, b} F(w, b) = \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^n C_i \max(0, 1 - \tilde{y}_i(w^\top x_i + b))^2$$

10.10.1 Gradient and Generalized Hessian

Letting $\mathcal{I}(w, b) = \{i \mid 1 - \tilde{y}_i(w^\top x_i + b) > 0\}$ be the set of observations violating the margin, the gradient of F with respect to w is expressed as Fan2008:

$$\nabla_w F(w, b) = w + 2X_{\mathcal{I}}^\top C_{\mathcal{I}}(X_{\mathcal{I}}w + b e_{\mathcal{I}} - \tilde{y}_{\mathcal{I}})$$

The function F is of class C^1 , but the $\max(0, \cdot)^2$ operator introduces a discontinuity in the second derivative directly at the margin boundary where $1 - \tilde{y}_i(w^\top x_i + b) = 0$. To address this non-differentiability, the generalized Hessian in the Clarke sense is used by differentiating the active linear terms of the gradient:

$$\nabla_w^2 F(w, b) = I_{92} + 2X_{\mathcal{I}}^\top D_{\mathcal{I}} X_{\mathcal{I}}$$

where $D_{\mathcal{I}} \in \mathbb{R}^{|\mathcal{I}| \times |\mathcal{I}|}$ is a diagonal indicator matrix containing the penalties C_i for violating observations, effectively mapping to the sum of outer products $\sum_{i \in \mathcal{I}} C_i x_i x_i^\top$.

Because each $C_i > 0$, the term $2X_{\mathcal{I}}^\top D_{\mathcal{I}} X_{\mathcal{I}}$ forms a positive semi-definite matrix. The presence of the identity matrix I_{92} acts as a full-rank regularizer, shifting the eigenvalues away from zero. This guarantees that the generalized Hessian is strictly positive definite ($\nabla^2 F \succ 0$), which eliminates saddle points, ensures strict local convexity within the optimization steps, and guarantees rapid convergence to the unique global minimum.

10.10.2 Principle of TRON (*Trust Region Newton Method*)

The TRON algorithm iteratively solves this unconstrained problem without requiring direct Hessian inversion. It relies on two principles:

1. **Trust Region:** the step search is confined to a perimeter of radius Δ_t where the local quadratic approximation is deemed reliable.
2. **Truncated Conjugate Gradient (CG):** the Newton step is calculated without explicitly instantiating the Hessian in memory, evaluating only the Hessian-vector product:

$$\nabla^2 F(w, b) \cdot s = s + 2X_{\mathcal{I}}^\top (D_{\mathcal{I}}(X_{\mathcal{I}}s))$$

At each iteration t , TRON solves the constrained quadratic subproblem:

$$\min_s q_t(s) = (g_t)^\top s + \frac{1}{2} s^\top \nabla^2 F(w_t, b_t) s \quad \text{s.t.} \quad \|s\|_2 \leq \Delta_t$$

and then performs the update $w_{t+1} = w_t + s_t$.

10.10.3 Convergence Guarantees

In accordance with the work of Lin, Weng, and Keerthi (2008), the sequence $\{w_t\}$ converges globally to the unique optimum w^* . The convergence is initially linear during the exploration phase, then becomes **quadratic** near the optimum:

$$\lim_{t \rightarrow \infty} \frac{\|w_{t+1} - w^*\|_2}{\|w_t - w^*\|_2} = 0$$

This dual property validates the algorithmic choice of LIBLINEAR in primal mode, reconciling the robustness of the SVM with the speed required to process the 123,169 observations of RT-IoT2022.

11 Theoretical Formulation of the RBF SVM (Non-linear)

The relatively long training time observed for LinearSVC in our experiment should not be interpreted as a direct consequence of linearity itself. It is more likely related to solver behavior, conditioning of the preprocessed data, convergence tolerance, the multi-class one-vs-rest strategy, and the specific hyperparameter configuration. The linear SVM remains theoretically more scalable in high-sample regimes, but empirical runtime depends strongly on implementation details. The linear SVM thus seeks to find an hyperplane corresponding to the model's decision boundary in the input space:

$$H = \{x \in \mathbb{R}^n : w^T x + b = 0\} [13]$$

In a real-world scenario like analyzing RT-IoT2022 traffic, boundaries distinguishing normal traffic from a cyberattack are unfortunately very rarely linear. Classes are often mixed together, which is why the RBF (*Radial Basis Function*) kernel SVM is utilized.

To solve this problem without exploding computational complexity, the RBF SVM modifies the approach along two major axes:

- **The Kernel Trick:** Instead of trying to deform the hyperplane in the 92-dimensional space, the model implicitly projects data into a higher-dimensional feature space. In this new space, mixed data becomes linearly separable. The flat hyperplane drawn there corresponds, once brought back to the original space, to a curved and complex decision boundary capable of accurately enveloping different attack typologies.
- **Measuring Local Similarity with the RBF Function:** The RBF kernel no longer just looks at which side of a line a point sits, it evaluates the proximity or similarity between two observations. Mathematically, it is expressed by:

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

The closer the characteristics of two network frames, the more the function's value will tend toward 1. Conversely, the further apart their characteristics, the more it will tend toward 0. The parameter γ controls the radius of influence of each support vector.

Thus, the objective function of this model remains algebraically very similar to that of the linear SVM. However, the dot product is replaced by the RBF kernel function. The optimization challenge then becomes finding the balance point between the margin C and the separation boundary γ to maximize threat detection.

11.1 Quadratic Optimization Problem

The goal of the standard soft-margin SVM (L1-SVM) is to maximize the margin while linearly penalizing errors via slack variables ξ_i .

In its primal form, the problem is written as:

$$\min f(w, b, \xi) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

under the constraints:

$$\begin{aligned} g_i(w, b, \xi) &= 1 - \xi_i - y_i (w^T \phi(x_i) + b) \leq 0 \\ -\xi_i &\leq 0 \end{aligned}$$

Where:

- $\phi(x_i)$: the data projection in the feature space.

- n : number of samples in the RT-IoT2022 dataset.
- $x_i \in \mathbb{R}^{92}$: feature vector.
- $y_i \in \{-1, +1\}$: target variable.
- $C = 1.0$: regularization parameter.
- γ : RBF kernel parameter.

11.2 Solving the Problem

11.2.1 Verifying Convexity

To guarantee that the Karush-Kuhn-Tucker conditions will lead us to a unique global minimum, we must prove that the problem is indeed convex:

- **The objective function** $f(w, b, \xi)$: The squared norm $\frac{1}{2}\|w\|^2$ is a strictly convex function while the penalty function $C \sum_{i=1}^n \xi_i$ is a linear affine function. The sum of a strictly convex function and an affine function (with $C > 0$) is a strictly convex function.
- **The constraints** g_k : Affine functions are convex, so the feasible domain defined by the inequalities is a convex set.

We can therefore see that the problem is an initially convex problem with affine function constraints.

11.2.2 Verifying Slater's Condition

The Slater condition is satisfied if there exists a point $x^* \in \mathbb{R}^n$ such that [13]:

$$\begin{aligned} g_i(x^*) &< 0, \quad i = 1, \dots, m \\ h_j(x^*) &= 0, \quad j = 1, \dots, r \end{aligned}$$

Slater's condition ensures the existence of an optimal solution by verifying there is a point strictly inside the constraint domain. We must therefore find (w, b, ξ) such that $g_k < 0$.

Let us arbitrarily fix $w_0 = 0 \in \mathbb{R}^{92}$ and $b_0 = 0 \in \mathbb{R}$. The constraints then become:

$$\begin{aligned} 1 - \xi_i &< 0 \implies \xi_i > 1 \\ -\xi_i &< 0 \implies \xi_i > 0 \end{aligned}$$

It is then enough to choose a value of $\xi_i > 1$ for the inequalities to be strictly respected. Slater's condition is therefore satisfied. The KKT conditions can then be used.

11.2.3 The Lagrangian

The Lagrangian takes the following formula:

$$L(x, \lambda, \mu) = f(x) + \lambda^T g(x) + \mu^T h(x) [13]$$

We introduce:

- $\alpha_i \geq 0$ for classification constraints.
- $\beta_i \geq 0$ for $\xi_i \geq 0$ constraints.

We can then construct our Lagrangian as follows:

$$L(w, b, \xi, \alpha, \beta) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y_i (w^T \phi(x_i) + b)) - \sum_{i=1}^n \beta_i \xi_i$$

The term $-\sum_{i=1}^n \beta_i \xi_i$ is obtained starting from the constraint $\xi_i \geq 0$:

$$\begin{aligned} -\xi_i &\leq 0 \\ \sum_{i=1}^n \beta_i (-\xi_i) &\leq 0 \implies -\sum_{i=1}^n \beta_i \xi_i \end{aligned}$$

11.2.4 KKT Conditions

Stationarity Condition Derivative with respect to w :

$$\nabla_w L = w - \sum_{i=1}^n \alpha_i y_i \phi(x_i) = 0 \implies w^* = \sum_{i=1}^n \alpha_i y_i \phi(x_i)$$

Derivative with respect to b :

$$\nabla_b L = -\sum_{i=1}^n \alpha_i y_i = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0$$

Derivative with respect to ξ_i :

$$\nabla_{\xi_i} L = C - \alpha_i - \beta_i = 0 \implies \alpha_i + \beta_i = C$$

Primal Feasibility

$$\begin{aligned} y_i (w^{*T} \phi(x_i) + b^*) &\geq 1 - \xi_i^* \\ \xi_i^* &\geq 0 \end{aligned}$$

Dual Feasibility The Lagrange multipliers must be positive or zero:

$$\begin{aligned} \alpha_i &\geq 0 \\ \beta_i &\geq 0 \end{aligned}$$

Complementary Slackness The product of each multiplier by its associated constraint must be zero at the optimal point:

$$\begin{aligned} \alpha_i (1 - \xi_i^* - y_i (w^{*T} \phi(x_i) + b^*)) &= 0 \\ \beta_i \xi_i^* &= 0 \end{aligned}$$

11.2.5 Transition to the Dual Problem and RBF Kernel Justification

To solve the dual problem, we must inject the stationary points into the Lagrangian, which gives us the Dual problem to maximize:

$$\text{Max } D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \phi(x_i)^T \phi(x_j)$$

Under the constraints:

$$0 \leq \alpha_i, \alpha_j \leq C$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

This is where the kernel trick appears: Mercer's theorem allows replacing the dot product $\phi(x_i)^T \phi(x_j)$ with the RBF kernel function:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

It is important to know that Mercer's theorem is usable because K is symmetrical and positive semi-definite. We then obtain:

$$D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

11.2.6 Problem Encountered with the RBF

The RBF kernel SVM is a powerful model to use for modeling non-linear relationships. Its theoretical efficiency is accompanied by major computational limits. These limitations become critical in the context of studying large-scale and high-dimensional datasets, notably our RT-IoT2022 dataset originally containing 123117 rows and 85 columns.

The RBF kernel SVM relies on a dual formulation of the optimization problem where data never interacts directly in an explicit feature space but only through a kernel function:

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

This formulation bypasses constructing a projection space, which is potentially infinite-dimensional. However, all interactions between observations must be evaluated pairwise. The RBF SVM optimization relies on the Gram matrix:

$$K \in \mathbb{R}^{n \times n}, \quad K_{ij} = K(x_i, x_j)$$

Thus, as soon as the number of samples increases, the problem's structure becomes quadratically memory-intensive with a complexity of $\mathcal{O}(n^2)$. In our study's context, $n = 123117$, which implies approximately $n^2 \approx 15.2 \times 10^9$ entries, equating to roughly 120 GB of RAM for a computer.

11.2.7 The SMO (Sequential Minimal Optimization) Method [14]

Although the RBF kernel SVM offers great expressiveness for non-linear problems, its resolution relies on a quadratic optimization problem whose complexity becomes prohibitive for large datasets due to the Gram matrix of size $\mathcal{O}(n^2)$. To make this problem solvable, decomposition algorithms like SMO (Sequential Minimal Optimization) are used.

These methods avoid globally solving the quadratic problem by optimizing only a subset of dual variables at each iteration, which allows for a significant reduction in computational and memory costs.

Because of the equality constraint $\sum \alpha_i y_i = 0$, it is impossible to modify only one variable without violating the rule. The first step of SMO is then to reduce our Dual problem to a 2-variable problem while fixing the other $N - 2$ multipliers $(\alpha_3, \dots, \alpha_N)$.

The equality constraint $\sum \alpha_i y_i = 0$ then reduces to a line equation:

$$\alpha_1 y_1 + \alpha_2 y_2 = - \sum_{i=3}^N \alpha_i y_i = \zeta$$

Where ζ is a known constant. By multiplying by y_1 (knowing that $y_1^2 = 1$), we can express α_1 in terms of α_2 :

$$\alpha_1 = y_1(\zeta - \alpha_2 y_2)$$

We can then substitute α_1 within the function $D(\alpha)$, giving us a quadratic function with a single unknown. Since the kernel matrix K is normally of size $n = 123, 117$, by fixing 2 variables, we obtain the Gram matrix:

$$K = \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix}$$

It's important to note that the Gram matrix represents the similarity of points to each other. Meaning, K is a resemblance score. To illustrate, if we have a 2×2 matrix, then: K_{11} can be translated as: "how much point 1 resembles point 1", which is 1.

$$D(\alpha_2) = -\frac{1}{2}\eta\alpha_2^2 + \kappa\alpha_2 + C_0$$

Where:

- $\eta = K_{11} + K_{22} - 2K_{12}$: represents the "distance" between our two points in the new complex space.
- κ : groups the fixed points and terms dependent on the constraint ζ .
- C_0 : terms not containing α_2 .

Since the dual function is concave, its maximum is reached when its derivative cancels out:

$$\begin{aligned} \frac{\partial D(\alpha_2)}{\partial \alpha_2} &= 0 \\ -\eta\alpha_2 + \kappa &= 0 \end{aligned}$$

This gives us a possible solution of $\alpha_2^* = \frac{\kappa}{\eta}$.

We can also represent α_2^* by:

$$\alpha_2 = \alpha_2^{old} + \frac{y_2(E_1 - E_2)}{\eta}$$

$E_i = f(x_i) - y_i$ represents the SVM's classification error for observation x_i .

The dual problem imposes that $0 \leq \alpha_2 \leq C$. To guarantee that updating α_2 does not force the first multiplier α_1 out of these bounds, SMO will calculate strict lower limit L and upper limit H based on the labels y_1 and y_2 (knowing whether the traffic is normal or not).

Geometrically, L and H guarantee that during the calculation, we do not exit the area permitted by penalty C [15]:

If $y_1 \neq y_2$:

$$\begin{aligned} L &= \max(0, \alpha_2^{old} - \alpha_1^{old}) \\ H &= \min(C, C + \alpha_2^{old} - \alpha_1^{old}) \end{aligned}$$

If $y_1 = y_2$:

$$\begin{aligned} L &= \max(0, \alpha_1^{old} + \alpha_2^{old} - C) \\ H &= \min(C, \alpha_1^{old} + \alpha_2^{old}) \end{aligned}$$

$$\alpha_2^{new} = \begin{cases} L & \text{if } \alpha_2^* < L \\ \alpha_2^* & \text{if } L \leq \alpha_2^* \leq H \\ H & \text{if } \alpha_2^* > H \end{cases}$$

With a new α_2^{new} , it is then important to change α_1 , which gives us:

$$\begin{aligned}\alpha_1 &= y_1(\zeta - \alpha_2 y_2) \\ \alpha_1^{new} &= \alpha_1^{old} + y_1 y_2 (\alpha_2^{old} - \alpha_2^{new})\end{aligned}$$

Since the weights carried by α_1 and α_2 have changed, the orientation of the decision boundary has also changed. It is therefore mandatory to recalculate the intercept value b so that the KKT conditions are perfectly respected for the two network samples just optimized.

The algorithm will then compute two potential values, b_1 and b_2 [15]:

If $0 < \alpha_1^{new} < C$ (which means point 1 is on the margin):

$$b_1 = b^{old} - E_1 - y_1(\alpha_1^{new} - \alpha_1^{old})K(x_1, x_1) - y_2(\alpha_2^{new} - \alpha_2^{old})K(x_1, x_2)$$

If $0 < \alpha_2^{new} < C$ (which means point 2 is on the margin):

$$b_2 = b^{old} - E_2 - y_1(\alpha_1^{new} - \alpha_1^{old})K(x_1, x_2) - y_2(\alpha_2^{new} - \alpha_2^{old})K(x_2, x_2)$$

The sign difference between the study and the current equalities is explained by the use of $E_i = f(x_i) - y_i$, which turns the equality into a subtraction.

The program then chooses the new value b^{new} according to the following rules:

- If α_1^{new} is valid on the margin, $b^{new} = b_1$.
- If α_2^{new} is valid on the margin, $b^{new} = b_2$.
- If both are on the margin, $b_1 = b_2$, $b^{new} = b_1$.
- If both multipliers have hit the exact bounds (0 or C), the algorithm takes the average: $b^{new} = \frac{b_1 + b_2}{2}$.

Now that α and b have changed, the program must then recalculate and update the estimation errors $E_i = f(x_i) - y_i$.

Finally, the program will do this for all multipliers α_i . If all data validate KKT, then the algorithm stops. If there remain points violating the KKT conditions, SMO selects data vectors that most violate the KKT conditions and restarts the analytical process. The idea behind this is to focus the effort where the model is most wrong to correct the decision boundary as quickly as possible.

Our python program `SVM(kernel="rbf")` must run on a huge amount of data in the real world. The LIBSVM library will then add three optimizations to the SMO we just discussed [16]:

- **Kernel Caching:** SMO avoids storing a 120 GB matrix in our case, but LIBSVM will instead cache recent K_{ij} calculations in our computer's memory to avoid endlessly recomputing the same RBF kernel exponentials.
- **Second Order Information:** SMO indeed selects data vectors violating KKT conditions the most, but LIBSVM uses a second-order heuristic to choose the α pair that will decrease the overall error the fastest at each iteration.
- **Shrinking:** As the algorithm runs, many alphas will get stuck at either 0 or C, and will then be pulled out of the iteration loop, allowing it to go from 123,117 variables to, for example, 12,000 variables after a few iterations, which drastically lowers the final execution time.

Conclusion

The objective of this project was to apply convex optimization concepts learned in class to a concrete and useful machine learning problem: intrusion detection in IoT networks using the RT-IoT2022 dataset. Our study compared two SVM approaches, a Linear SVM and an RBF kernel SVM, not only to evaluate their classification performance, but also to analyze the optimization problems actually solved by these models.

From an experimental point of view, the RBF SVM obtained the best overall detection performance. On the test set, it achieved an accuracy of 0.9952 and a macro F1-score of 0.9490, compared with 0.9913 accuracy and 0.9431 macro F1-score for the Linear SVM. This confirms that the RBF kernel is relevant for this dataset: by implicitly projecting the data into a higher-dimensional feature space, it can model non-linear decision boundaries and better separate some complex or minority traffic classes.

However, the Linear SVM remains highly competitive. Despite its simpler decision function, it reaches performance very close to the RBF model. Its main strength lies in its direct link with convex optimization: it relies on margin maximization, regularization, slack variables, and clearly analyzable optimality conditions. It therefore provides the most natural bridge between machine learning experimentation and the mathematical concepts studied in class.

The two models also differ strongly from an operational perspective. In our experiment, the RBF SVM was surprisingly faster to train than the Linear SVM, which can be explained by implementation and solver behavior on this specific dataset. Nevertheless, the situation is reversed at inference time. The Linear SVM requires only 0.000877 ms per sample, whereas the RBF SVM requires 0.149077 ms per sample, making the RBF model approximately 170 times slower in prediction. This difference is essential for a real-time IDS, where each millisecond may matter when analyzing continuous network flows.

The answer to our initial problem is therefore nuanced. If the priority is to maximize detection quality, especially on non-linear patterns and certain minority classes, the RBF SVM is the most relevant choice in our experiment. If the priority is real-time deployment, interpretability, simplicity, and fast inference with only a small loss in performance, the Linear SVM offers a better compromise.

This study nevertheless presents several limitations. The dataset is highly imbalanced, making accuracy insufficient to evaluate the models properly. Some rare classes contain very few test examples, so their per-class scores must be interpreted cautiously. The results also depend on the train/test split, the chosen hyperparameter grid, the execution environment, and the specific scikit-learn implementations used.

Thus, the main contribution of this project is not to declare a universal winner between Linear SVM and RBF SVM, but to show that model selection depends on the criterion being prioritized. The RBF SVM is more performant in our experiment, while the Linear SVM remains more interpretable, theoretically more scalable, and much faster at inference. Within the scope of a convex optimization project, this tension between empirical performance, computational cost, and mathematical structure constitutes the central contribution of our study.

References

- [1] B. S. Sharmila and R. Nagapadma, RT-IoT2022, UCI Machine Learning Repository, 2024. DOI: 10.24432/C5P338.
- [2] B. S. Sharmila and R. Nagapadma, 'Quantized autoencoder (QAE) intrusion detection system...', Cybersecurity, 2023. DOI: 10.1186/s42400-023-00178-5.
- [3] B. S. Sharmila et al., 'Performance Evaluation of Parametric and Non-Parametric Machine Learning Models...', JSIR, 2024. DOI: 10.56042/jsir.v83i8.7437.
- [4] P. Dymora et al., 'Comparative analysis of machine learning models for multiclass anomaly detection...', ASTRJ, 2026. DOI: 10.12913/22998624/213807.
- [5] C. S. Kumar et al., 'IoT Intrusion Detection with Mutual Information Feature Selection...', CANA, 2025. DOI: 10.52783/cana.v32.5821.
- [6] scikit-learn developers, 'sklearn.svm.SVC - C-Support Vector Classification.'
- [7] scikit-learn developers, 'Support Vector Machines - Complexity,' User Guide.
- [8] C. Cortes and V. Vapnik, 'Support-vector networks,' Machine Learning, 1995.
- [9] R.-E. Fan et al., 'LIBLINEAR: A Library for Large Linear Classification,' JMLR, 2008.
- [10] Internal group document, 'Comparative table of works with SVM on RT-IoT2022,' 2026.
- [11] Course scoping document, 'Convex Optimization Project 2026,' 2026.
- [12] Internal group notebook, "Part 1 - experimental study: comparison Linear SVM vs RBF SVM on RT-IoT2022", executed version, 2026.
- [13] Convex Optimization Course.
- [14] <https://pages.hmc.edu/ruye/MachineLearning/lectures/ch9/node9.html>
- [15] <https://bitmask93.github.io/ml-blog/Sequential-Minimal-Optimization-for-Support-Vector-Machines/>
- [16] <https://www.csie.ntu.edu.tw/~cjlin/papers/libsvm.pdf>